

FEDERAL AUTONOMOUS EDUCATIONAL
INSTITUTION OF HIGHER PROFESSIONAL EDUCATION
«NATIONAL RESEARCH UNIVERSITY
«HIGHER SCHOOL OF ECONOMICS»

Faculty of Informatics, Mathematics, and Computer Science

Novikova Irina Alekseevna

**DEVELOPMENT OF A NEW METHOD FOR LLM FINE-TUNING BASED
ON IMPORTANCE OF BUILDING BLOCKS**

Diploma

in the field of 01.04.02 Applied Mathematics and Informatics

educational program

"Data Mining"

Academic Supervisor
PhD in Computer Science
A.V. Demidovskij

Nizhny Novgorod, 2026

Contents

Introduction	4
Chapter 1. Theoretical foundations of fine-tuning	
acceleration through importance analysis	7
1.1 Definition of Transformer-based architecture	7
1.2 Key directions in Large Language Models fine-tuning	
acceleration	8
1.2.1 Data reduction	9
1.2.2 Parameter-Efficient Fine-Tuning	9
1.2.3 Quantization	11
1.2.4 Model pruning	12
1.2.5 Backpropagation elimination	13
1.3 Existing approaches for assessing the importance within	
Large Language Models	14
1.3.1 Importance of separate weights	15
1.3.2 Importance of layers	18
1.3.3 Importance of Transformer blocks	20
1.3.4 Importance of predicted architectures	22
1.4 Overview of considered competitors	23
Chapter 2. Development of an approach to accelerate the fine-tuning through	
importance analysis	25
2.1 Importance-based Transformer blocks selection	26
2.2 Convergence-based Transformer blocks selection	28
2.3 Key differences from existing approaches	29
Chapter 3. Evaluation of the proposed method	30
3.1 Training and evaluation protocols	30

3.2	Analysis of the obtained results	32
3.3	Sensitivity analysis	34
	Conclusion	37
	Reference List	39

Introduction

The **relevance** of this work is determined by the pivotal role that Large Language Models (LLMs) currently play in various domains including machine learning [86], computer vision [90], and natural language processing [8]. These pre-trained LLMs are actively employed in numerous industrial applications, prized for their proficiency in tasks such as in-context learning [8], instruction following [70], and structured reasoning [89]. However, the capabilities of a pre-trained LLM may not be sufficient to tackle more specialized tasks, for example, in medical or scientific domains. This necessitates extensive fine-tuning to tailor the models for specific downstream tasks. However, the fine-tuning process is extremely resource-consuming and demands significant memory and time resources, resulting in low training throughput [62]. This way, the evolvement of novel fine-tuning strategies that reduce time and memory costs without sacrificing model quality is of great importance.

At the same time, given the vast parameter space of modern LLMs, which results in a significant over-parameterization [17], identifying and leveraging the most essential model structures becomes crucial. These structures are vital as they disproportionately impact model performance [94]. Effective utilization of LLMs in various scenarios – whether pre-training, fine-tuning, or inference – demands a nuanced understanding of "importance" to discern these critical structures. This requirement extends across multiple optimization strategies, including Neural Architecture Search [20] (NAS), [83], [49], Parameter-Efficient Fine-Tuning (PEFT) [97], [71], quantization [54], [18], [43], and model pruning [26], [78], [82]. Each of these strategies aims to optimize different aspects of model architecture, yet they often rely on disparate heuristics for estimating importance, many of which lack direct correlation with model quality [20].

Therefore, there are numerous avenues for optimizing the use of LLMs, particularly through applying methodologies for assessing importance of structures within LLMs. However, to date, there have been no comprehensive studies that systematically investigate

methods for importance assessment across various structural levels. Addressing this gap is not only crucial for enhancing model efficiency but also for advancing the methodologies used in model optimization, in particular, in the field of model fine-tuning acceleration.

The **practical significance** of this research is that it introduces an innovative fine-tuning strategy for LLMs, which provides up to 63.4% of acceleration compared to state-of-the-art *LoRA* approach, thus enabling the further deployment of LLMs in resource-constrained environments.

The **novelty** of this work can be summarized as follows:

- This work introduces a comprehensive overview of the most prominent existing methodologies for importance and convergence estimation of different structures within the LLMs.
- This study introduces a new fine-tuning approach for LLMs, that dynamically assigns *LoRA* adapters during the fine-tuning based on importance and convergence analysis.

The **goal** of the present study is to develop a method for accelerating the fine-tuning of LLM while maintaining quality, with performance degradation compared to the baseline *LoRA* approach being less than 1%.

To achieve the goal, the following **objectives** were formulated:

1. Conduct a comprehensive background study on the topic of fine-tuning acceleration techniques.
2. Provide an overview of existing methodologies for estimating the importance of various structures within LLM and approaches to analyze the convergence of LLMs.
3. Propose a novel *LoRA*-based fine-tuning strategy.

4. Offer a detailed evaluation of the proposed method and its hyperparameters among various LLMs.
5. Conduct a comparative analysis between the proposed solution and the closest competitors in the field.
6. Analyze obtained results and draw conclusion.

The study employs a combination of **methodological approaches** to address the research objectives. Firstly, it adopts a method of theoretical analysis and synthesis of existing literature to establish a solid theoretical foundation for the research. Secondly, programming methods are utilized to analyze the importance and convergence of LLMs'. Lastly, the study employs Deep Learning techniques to fine-tune LLMs.

The **object** of this study is the fine-tuning acceleration of LLMs through importance analysis of Transformer blocks.

This study was conducted as part of a **research project at the Huawei** dedicated to the fine-tuning acceleration of LLMs. The results of this study were presented at the European Conference on Artificial Intelligence 2025 (ECAI, Rank A) and Neuroinformatics 2024, and published in the articles "On the Influence of Layer Importance on LLM Fine-Tuning Acceleration and Quality" and "Comprehensive Weight Decomposition Analysis of Modern Parameter-Efficient Methods" in journals indexed in Scopus (Q4 and Q3), respectively. In the article "On the Influence of Layer Importance on LLM Fine-Tuning Acceleration and Quality" my individual contributions included writing the introduction, overview of the existing solutions in the field, conducting and analyzing the main experiments, as well as writing the conclusion and outlining the current limitations of the work. In the article "Comprehensive Weight Decomposition Analysis of Modern Parameter-Efficient Methods" my individual contributions consisted of writing the introduction, reviewing the current state-of-the-art in the field, implementing the methodology, conducting and analyzing the main experiments, and writing the conclusion.

Chapter 1. Theoretical foundations of fine-tuning acceleration through importance analysis

The goal of the present study is to develop a method for accelerating the fine-tuning of Large Language Models. However, the development of a novel approach requires a comprehensive analysis of existing techniques and promising ideas in this field, which can be leveraged in future work. This chapter is dedicated to such a thorough analysis and review.

1.1 Definition of Transformer-based architecture

Let $\mathcal{M}_\Phi$ denote a Transformer autoregressive model with trainable parameters $\Phi \in \mathbb{R}^n$. The model $\mathcal{M}_\Phi$ is described with a set of Transformer blocks τ , where each block τ_i contains two submodules: a multi-head attention module (MHA) (1) and a module consisting of a Feed Forward Neural Network (FFNN) (2). Given the input sequence $x \in \mathbb{R}^n$, MHA module works with h parallel heads.

$$head_i = \text{Softmax}\left(\frac{xW_q(xW_k)^T}{\sqrt{d_h}}\right)xW_v \quad (1)$$

$$MHA(X) = \text{Concat}(head_1, \dots, head_h)W_o,$$

where $i \in \mathbb{R}$, $W_o \in \mathbb{R}^{d \times d}$ is an output projection, and $W_q, W_k, W_v \in \mathbb{R}^{d \times d_h}$ are query, key, and value projections of the i -th head; d_h is typically set to d/h . FFNN module consists of two linear transformations with a ReLU activation.

$$FFNN(x) = \text{ReLU}(xW_{up} + b_{up})W_{down} + b_{down}, \quad (2)$$

where $W_{up} \in \mathbb{R}^{d \times d}$ is the up-projection matrix and $W_{down} \in \mathbb{R}^{d \times d}$ is the down-projection matrix.

Within the LLM there exist a concept of the "importance" of certain structures. In this context, "importance" facilitates the identification of structures that can be

removed without affecting model performance. Thus, understanding the importance of a structure helps assess its impact on the model’s overall quality.

Fine-tuning is the process of turning a general-purpose model into a specialized one while updating all pre-trained model parameters over a downstream task dataset. During the fine-tuning process LLM parameters are updated until the loss function reaches approximately the optimum, indicating, that the model has been converged [1] and is ready for deployment.

Let fine-tuning data be organized as a dataset $D = \{(x_i, y_i), i = 1, \dots, N\}$, where x_i is a data sample, and y_i is a ground truth label; both x_i, y_i are sequences of tokens. Model prediction is denoted as $\mathcal{M}_\Phi(x_i)$. Fine-tuning assumes that the model is pre-trained: let it be initialized with pre-trained weights Φ_0 .

Full fine-tuning is the process of updating pre-trained weights to $\Phi_0 + \Delta\Phi$ so that the fine-tuning objective for $\mathcal{M}_\Phi$ is optimized over Φ (3).

$$\max_{\Phi} \sum_{(x,y) \in D} \sum_{t=1}^{|y|} \log(\mathcal{M}_\Phi(y_t|x, y_{<t})), \quad (3)$$

where $|y|$ is the number of tokens in the data sample, dimensionality of $|\Delta\Phi|$ is equal to $|\Phi_0|$ [35].

The resources required for full fine-tuning of a model are comparable to those needed for pre-training an LLM; the only difference is that the full fine-tuning necessitates a smaller amount of domain-specific data. Therefore, the development of approaches, that allow to reduce the number of required resources is of crucial importance.

1.2 Key directions in Large Language Models fine-tuning acceleration

Various modern approaches exist for reducing time and memory overhead during the full fine-tuning, including data reduction [22], Parameter-Efficient Fine-Tuning [35], quantization [19], model pruning [13], backpropagation elimination [63], [34].

1.2.1 Data reduction

Data reduction is the process of selectively removing less significant samples from a training dataset while maintaining the accuracy of the resultant model. This process encompasses three primary methodologies: dataset condensation or distillation, core-set selection, and importance sampling, each defined by its approach to create a compact version of the training dataset.

Dataset distillation involves synthesizing new samples based on statistical properties derived from the original dataset [9]. This method aims to condense the dataset into a smaller, yet representative, set of synthetic samples which effectively capture the underlying data distribution. Coreset selection facilitates the automatic identification and extraction of a subset from the original dataset [36]. This approach ensures that the core elements necessary for learning are retained, thereby optimizing the training process. Importance sampling focuses on selecting samples that contribute most significantly to the model’s learning, particularly those that induce larger updates to the model’s weights during training [37]. For example, ALOE [85] removes data samples based on losses obtained from a currently trained model or a pre-trained one, providing up to 92% of acceleration with quality degradation of 1.5% compared with *LoRA*. Moreover, these methods cause an additional overhead to the training process by requiring a full forward pass for all samples in a dataset [38].

1.2.2 Parameter-Efficient Fine-Tuning

Among all the aforementioned approaches, PEFT emerges as a leading method for accelerating the fine-tuning of LLMs. PEFT focuses on updating a minimal subset of the model’s parameters, significantly reducing time and memory costs, but preserving or even enhancing model accuracy [30]. In the context of PEFT, $\Delta\Phi$ is encoded by a much smaller-sized set of parameters Θ , so that $|\Theta| \ll |\Phi_0|$, $\Delta\Phi = \Delta\Phi(\Theta)$. Thus, PEFT aims at maximizing the fine-tuning objective over Θ (4).

$$\max_{\Theta} \sum_{(x,y) \in D} \sum_{t=1}^{|y|} \log(\mathcal{M}_{\Phi_0 + \Delta\Phi(\Theta)}(y_t|x, y_{<t})) \quad (4)$$

LoRA [35] is de-facto standard approach in the industry, that freezes the pre-trained model weights and injects trainable rank decomposition matrices into each layer of the Transformer architecture, greatly decreasing the number of trainable parameters for downstream tasks. Let $W_0 \in \mathbb{R}^{d_1 \times d_2}$ denote pre-trained weights of an arbitrary layer, x is an input data. Then, inference of this layer, denoted as h can be expressed as $h = W_0 x$. Applying *LoRA* to that layer modifies its forward pass as follows (5).

$$h = W_0 x + \Delta W_x = W_0 x + B A x, \quad (5)$$

where $W_0, \Delta W \in \mathbb{R}^{d_1 \times d_2}$, $A \in \mathbb{R}^{r \times d_2}$, $B \in \mathbb{R}^{d_1 \times r}$, $r \ll (d_1, d_2)$. A, B matrices are called *LoRA* adapters.

By greatly decreasing the number of model parameters, *LoRA* provides up to 50% memory reduction and 150% acceleration compared to full fine-tuning. Although *LoRA* is a state-of-the-art approach, it still has several drawbacks. First, *LoRA* lacks the nuanced capability for more subtle weight adjustment during the fine-tuning [56]. Second, *LoRA* adapters' initialization approach could lead to a slower model convergence [67]. Third, *LoRA* overlooks the varying importance of different weight matrices [97]. Therefore, DoRA [56], PiSSA [67], and AdaLoRA [97] are deemed promising improvements of *LoRA*.

DoRA [56] approach suggests decomposing the pre-trained weights W_0 into *magnitude* and *direction* components (6). *Magnitude* is a trainable vector $M = \|W_0\|_c$, which represents a vector-wise norm of a matrix across each column. *Direction* is the ratio of the pre-trained weight matrix to its norm. The *direction* component is updated using *LoRA* [35].

$$W = M \frac{W_0 + B A}{\|W_0 + B A\|_c} \quad (6)$$

DoRA provides up to 3% of quality improvement on the fine-tuning of LLaMA2-7B and LLaMA2-13B compared with *LoRA*.

PiSSA [67] approach modifies the *LoRA* by initializing A and B adapter matrices with the principal components of the original matrix W computed utilizing Singular Value Decomposition. Thus, trainable adapters contain the most essential directions of W , that is why training adapters mirror the process of the entire model fine-tuning. *PiSSA* provides up to 21.05% of quality improvement on LLaMA2-7B fine-tuning compared with *LoRA*.

AdaLoRA [97] approach adaptively allocates the parameter budget among the weight matrices according to their importance score. It re-parametrizes the incremental updates in the form of Singular Value Decomposition and modifies the *LoRA* forward pass as follows (7).

$$h = W_0x + \Delta W_x = W_0x + P\Lambda Q, \quad (7)$$

where $P \in \mathbb{R}^{d_1 \times r}$ and $Q \in \mathbb{R}^{r \times d_2}$ represent singular vectors of ΔW , the diagonal matrix $\Lambda \in \mathbb{R}^{r \times r}$ contains the singular values $\{\lambda_i\}_{1 \leq i \leq r}$. *AdaLoRA* provides 1% of quality improvement on DeBERTa-V3-base fine-tuning compared with *LoRA*.

1.2.3 Quantization

Quantization is a technique used to transform high-precision data into low-precision data. This process is predominantly utilized to accelerate inference, but recent studies have also explored its application in enhancing the efficiency of the fine-tuning.

One notable approach is QLoRA [19], which introduces a 4-bit NormalFloat data type for storing frozen pre-trained language model while maintaining computational operations, such as matrix multiplication of *LoRA* adapters, in 16-bit BrainFloat precision. Despite achieving a substantial memory reduction, this method provides slowdown compared to standard *LoRA* approach due to quantization and dequantization of LLM weights. In addition, when applying the extreme low-bit quantization, the

huge quantization error can adversely impact the initialization of *LoRA* fine-tuning [30]. Another method, PEQA [42], focuses on fine-tuning the quantization scales of quantized LLMs, while keeping the integer matrices unchanged. This approach results in up to 92% memory reduction, however introduces a meaningful quality degradation of 5% compared to *LoRA*. Additionally, AlphaTuning is a technique that involves quantizing a pre-trained LLM and fine-tuning certain quantized parameters for specific tasks [47]. This method not only accelerates fine-tuning by 38% but also reduces memory usage by 80% with a quality decrease of at least 5%.

1.2.4 Model pruning

Model pruning is a technique aimed at reducing the model by eliminating redundant parameters or substructures. This process is primarily categorized into three approaches: unstructured, semi-structured, and structured pruning [13].

Unstructured pruning removes individual parameters based on importance criterion, achieving high theoretical sparsity and minimal quality degradation. SOTA methods in this area reports less than 5% quality drop at 50% unstructured sparsity. However, its irregular memory access complicates hardware acceleration and performance often matches or falls bellow dense counterparts due to overheads in parameter managements.

Structured pruning targets entire model components (e.g. channels, layers, Transformer blocks), making it hardware-friendly, but causing significant quality degradation due to removal of large parameter groups. SOTA methods in this area provide up to 13% quality degradation at 50% structured weight sparsity.

Semi-structured pruning removes N out of every M consecutive weights and thus offers a favorable compromise between accuracy and hardware efficiecnecy across various GPU architectures. However, SOTA approaches in this field still provides up to 5% quality degradation under 2:4 sparsification pattern.

Moreover, with weight pruning the degradation in model performance extremely increases as the proportion of pruned parameters grows. In addition, pruning process

often introduces computational overhead on additional inference on calibration data or further training to recover the LLM performance.

1.2.5 Backpropagation elimination

Backpropagation elimination involves replacing the traditional backpropagation method with alternative learning rules that are less computationally expensive. The primary motivation behind this is to achieve significant reductions in both memory and time requirements. This scientific field is represented by two main directions: zeroth-order methods and localized learning strategies.

Zeroth-order methods estimate gradients using only two forward passes. A notable advancement in this category is the Memory-efficient Zeroth-order Optimizer (MeZO) [63], that efficiently implements the ZO-SGD method [80]. It has been shown to deliver performance on par with traditional backpropagation fine-tuning across various tasks. Notably, MeZO achieves up to a twelvefold reduction in memory use and up to a twofold decrease in training time. However, it requires considerably more iterations than conventional fine-tuning and only reaches acceptable levels of accuracy when combined with a specifically tailored prompt [63].

The second category involves localized learning rules, particularly the Hebbian rule. The Hebbian rule [7] draws inspiration from the structural and functional organization of the human brain. It suggests that the strength of synaptic connections between neurons increases when the neurons activate simultaneously and weakens when they do not. An efficient combination of localized learning rule with backpropagation provides an acceleration by up to 77%, however introduces a substantial quality degradation of 3.04% on average [16].

Therefore, there exist various approaches for fine-tuning acceleration.

- **Data reduction** techniques provide significant acceleration by reducing the size of the training dataset, lowering the computational resources required for model training. However, these methods introduce extra overhead during the

training process and leads to quality degradation due to removal of training samples from the dataset.

- **Quantization** strategies significantly reduce both the time and memory required for fine-tuning, they generally result in a noticeable decline in model quality.
- **Model pruning** techniques provide substantial acceleration and memory reduction, however lead to a quality drop, bring time overhead on additional inference on calibration data or retraining, and demand special hardware for efficient computation.
- **Backpropagation elimination** methods significantly reduce computational demand by lowering memory and training time, but often require more iterations to achieve satisfactory accuracy or introduce quality degradation compared to backpropagation-only approaches.

Hence, Parameter-Efficient Fine-Tuning is considered the most prominent approach, since its state-of-the-art *LoRA* method provides noticeable acceleration of up to 150% with memory reduction of 50% maintaining the quality comparable to full fine-tuning. Moreover, significant optimization opportunities of *LoRA* are presented by selectively attaching adapters to a subset of the important Transformer blocks.

1.3 Existing approaches for assessing the importance within Large Language Models

The evaluation of importance can be implemented for various structures, ranging from individual weights to Transformer blocks, and even entire model architectures. The overview of observed methods is presented in Table 1.

Table 1: Summary of considered approaches for importance estimation. Negative value is quality drop.

Structure	Reference	Best quality	Worst quality	Acceleration
Weight	[23]	1.7	–	N/A
	[11]	0.8	–	N/A
	[94]	-2.4	–	N/A
	[29]	-15.4	–	N/A
	[82]	-5.5	–	N/A
	[54]	-2.9	–	N/A
	[43]	2.2	–	N/A
	[45]	1.1	–	N/A
Layer	[96]	-19.5	–	N/A
	[88]	-2.4	–	N/A
	[55]	5.3	–	N/A
	[87]	4.1	–	N/A
	[32]	-5.7	–	N/A
	[84]	N/A	–	N/A
Transformer block	[66]	18.5	–	-14.1
	[51]	12.3	–	3.4
	[26]	N/A		-6.7
	[78]	27.6		-8.5
	[40]	23.3		-7.4
Model	[101]	N/A		4.2
	[20]	N/A		1.5

1.3.1 Importance of separate weights

The Lottery Ticket Hypothesis [23] suggests that within a randomly-initialized, dense neural network exists a smaller subnetwork, or the *winning ticket* which can achieve the same level of test accuracy as the full network if trained independently, often within the same or even fewer training iterations. This phenomenon has been observed in a range of architectures within both the Computer Vision and Natural Language Processing fields. For instance, a subnetwork within the LeNet architecture,

constituting only 51.3% of the original network’s weights, has demonstrated the ability to surpass the full model in terms of test accuracy more rapidly. Similarly, the *winning ticket* for the BERT-base model, albeit with a sparsity ratio of 90%, can perform within one standard deviation of the full model’s performance [11].

Developing the idea of the importance of individual weights within the model, the concept of systematic outliers in LLMs was introduced, which are defined as values that deviate significantly from the average of their distribution [2]. These outliers influence the model to predominantly predict the most frequent tokens, thereby affecting the overall quality of the model. Notably, the emergence of these outliers occurs in the Multi-Head Attention module of the Transformer block, significantly influenced by the SoftMax operation with dominant attention weights increasing exponentially for key values (8), while other weights approach zero (9). This imbalance in attention weights results in significant updates of the projection weights K and V during backpropagation. If the MHA module produces extreme values due to attention outliers, as the output from the MHA module passes through the MLP block, these anomalies are further amplified by the non-linear activation inside the MLP module and concentrated in the down-projection matrix. Thus, the combination of steep gradients and non-linear transformations fosters the emergence of outliers in both activations and weights.

$$A_{xj} = \frac{\exp(M)}{\exp(M) + (n - 1)} \approx 1 \quad (8)$$

$$A_{xk} = \frac{1}{\exp(M) + (n - 1)} \approx 0, \quad (9)$$

let $M = \frac{W_q(W_k^T)_j}{\sqrt{d_h}}, \frac{W_q(W_k^T)_k}{\sqrt{d_h}} = 0$, then $\frac{W_q(W_k^T)_j}{\sqrt{d_h}} \gg \frac{W_q(W_k^T)_k}{\sqrt{d_h}}$, n is the number of elements.

Various types of outliers were suggested [2], such as:

1. **Activation outliers inside layer.** For layer outputs $h_l \in \mathbb{R}^{B \times H}$ with B being the batch size, H being the hidden dimension, and $\mu_h = \frac{1}{B \cdot H} \sum_{i,j} |h_{i,j}|$ being the mean absolute value of layer outputs h_l , the set of activation outliers inside the Transformer block is defined as follows (10).

$$O = \{(i, j) \mid |h_{i,j}| > \tau \cdot \mu_h\}, \quad (10)$$

where τ is a predefined threshold.

2. **Activation outliers inside the down-projection weight.** For down projection in-puts $x_l^{down} \in \mathbb{R}^{B \times H}$ and $\mu_{x^{down}} = \frac{1}{B \cdot H} \sum_{i,j} |x_{i,j}^{down}|$, the set of activation outliers is defined as follows (11).

$$O = \{(i, j) \mid |x_{i,j}^{down}| > \tau \cdot \mu_{x^{down}}\} \quad (11)$$

3. **Weight outliers.** For projection weights $W \in \mathbb{R}^{O \times I}$ with O being output dimension, I being input dimension, and $\mu_{w_i} = \frac{1}{I} \sum_j |w_{i,j}|$ being the row-wise mean absolute value of W , the set of weight outliers is defined as follows (12).

$$O = \{(i, j) \mid |w_{i,j}| > \tau \cdot \mu_{w_i}\} \quad (12)$$

4. **Attention outliers.** For cumulative attention scores $A \in \mathbb{R}^{L \times L}$ with L being a sequence length, $\hat{A}_j = \sum_{i=1}^L A_{i,j}$ being the cumulative attention contribution for token j , and $\mu_A = \frac{1}{L} \sum_j \hat{A}_j$, the set of attention outliers is defined as follows (13).

$$O = \{j \mid \hat{A}_j > \tau \cdot \mu_A\} \quad (13)$$

These systematic outliers significantly impact model quality and pose challenges for model compression techniques, such as pruning and quantization [2].

Pruning In terms of pruning, the removal of weight outliers can have a devastating effect on model performance, leading to a quality degradation of up to 50% for LLaMA2-7B in few-shot learning tasks [94]. Conversely, preserving these crucial weights while eliminating the less important ones results in a quality degradation, not exceeding 2% [94].

However, even preserving these critical weights can still lead to quality drop. The magnitude pruning approach ([28], 14), which removes smallest absolute values

weights, sometimes results in suboptimal pruning and a degradation in a model performance on 15% for the LLaMA2-7B model [82].

$$i_{\xi} = |W|, \quad (14)$$

where $|W|$ is the absolute value of weight matrix.

Even more advanced methodologies such as Wanda [82] (15), which enhances sparsity by considering both weight magnitudes and input activations are still facing a similar problem. Despite demonstrating a 10% perplexity improvement for the LLaMA2-7B model compared to the traditional magnitude pruning, Wanda still leads to 5-15% of quality drop compared with the original dense model.

$$i_T = |W| \cdot \|X_T\|_2, \quad (15)$$

Quantization In terms of quantization, several strategies have been developed to account for the significance of individual weights. Outlier-aware quantization, for instance, has demonstrated superior performance, exceeding the state-of-the-art SmoothQuant [91] method by 1% on both the LLaMA2-7B and LLaMA2-13B models [94]. Moreover, skipping the quantization of these extremely important weights and maintaining them in the FP16 data format can enhance model quality by as much as 67% compared to their full quantization [54]. Additionally, sensitivity-based non-uniform quantization, which optimizes bit precision assignments and stores outliers and sensitive weight values in a space-efficient sparse format, results in less than 2% quality degradation when transitioning from a 16-bit to a 4-bit format [44].

1.3.2 Importance of layers

Moving forward, the significance of various layers within the Transformer model has been extensively studied, revealing intricate details about their functionalities and relative importance.

Previously, experiments on re-initialization and re-randomization robustness of Transformer layers inside LLMs were conducted [96]. This implies re-initializing or

assigning random parameters for different model layers, respectively, while keeping the rest of the model parameters intact. As a result, it was revealed that layers within these networks can be classified as either robust or critical: robust layers, when reset to their initial values, do not significantly impact the model’s performance. This suggests that not all layers are equally important, and some, especially in over-capacitated networks trained with stochastic gradient descent, are inherently less critical due to the model self-restricting complexity [96].

Besides, it was observed that individual Attention heads within the Transformer model serve distinct roles in the generation process [88]. Specifically, certain Attention heads are pivotal for identifying syntactic and dependency relationships, highlighting the specialized functions of these components within the architecture. This research also demonstrated that the encoder retains its efficacy despite a considerable reduction in the number of Attention heads, indicating a potential for optimization in the model design [88].

In a related vein, it was noted that perplexity – a measure of model performance – is more closely associated with overall model capacity than merely the presence of self-attention mechanisms [55]. This finding suggests that increasing the complexity of the model could potentially be more beneficial than focusing solely on the inclusion of self-attention layers.

Further expanding on the theme of layer importance, Tyukin et al. [87] reported that reducing the number of MHA layers by 40% leads to only a minimal decrease in performance. In contrast, a similar reduction in MLP layers results in a significantly greater decline in output quality of up to 60% for the LLaMA2-7B model. This disparity underscores the critical role of MLP layers in maintaining the overall effectiveness of the Transformer model.

Adding to the discourse, He et al. [32] discovered a high level of redundancy within MHA layers, suggesting that a substantial portion of these layers could be removed without notably impairing the model’s performance. This finding points towards potential efficiencies in model architecture by minimizing unnecessary layers.

Lastly, Tian et al. [84] elucidated the intricate relationship between attention logits and MLP weights, proposing that explicit updates to self-attention may be redundant. This is due to the implicit incorporation of self-attention functionalities in the lower MLP layers, further emphasizing the possibility of streamlining Transformer architectures without sacrificing performance.

1.3.3 Importance of Transformer blocks

Moving on to larger structures within LLMs, the saliency of the whole Transformer blocks could be estimated.

Various methodologies have been proposed to assess the importance of Transformer blocks in LLMs. Inspired by the aforementioned Wanda [82] approach, *OWS* [51] selectively prunes Transformer blocks based on their outlier distribution, stating that outliers mostly presented in the first and the last Transformer blocks. *OWS* allows to achieve a 12% reduction in memory usage and a modest 3% quality improvement over *LoRA*. However, *OWS*'s extensive parameters updates make it time-inefficient.

UIDL [26] states that layer-to-layer evolution of representations for a Transformer is given by a residual iteration equation (16).

$$x^{(l+1)} = x^{(l)} + f(x^{(l)}, \theta^{(l)}), \quad (16)$$

where $x^{(l)}$ is the multi-dimensional input, $\theta^{(l)}$ is parameters vector for layer l , $f(x, \theta)$ describes the transformation of one Transformer block.

To successfully prune the n layers before layer l it would be better if the input to the pruned block will be very similar to the output of the pruned block (17).

$$x^{(l)} = x^{(l-n)} + \varepsilon, \quad (17)$$

where ε is a constant indicating the degree to which one input may differ from another.

Therefore, the least important Transformer blocks could be defined as the least affecting on the model output and identified with cosine similarity between input and

output to the Transformer block (18).

$$i_T = \operatorname{argmin} \frac{1}{\pi} \arccos\left(\frac{X_T \cdot Y_T}{\|X_T\|_2 \cdot \|Y_T\|_2}\right), \quad (18)$$

where X_T and Y_T are the input and output to the Transformer block, respectively, $\|\cdot\|_2$ is a Frobenius norm.

The proposed approach allows to achieve a 40% reduction in layers without a quality drop for the LLaMA2-70B model [26]. However, *UIDL* requires additional inference on calibration data, which increases time overhead. Besides, cosine similarity was efficiently utilized in ShortGPT [66] leading to 18% of acceleration with quality drop of 14%.

Another approach is *SLEB* [78], which utilizes perplexity as a metric to evaluate Transformer block importance based on the assumption, that redundant blocks contribute less to the model’s outputs, and their removal leads to smaller degradation in perplexity (19).

$$i_T = \exp\left\{-\frac{1}{S} \sum \log p_{\theta}^n(X_T^{(S)} | X_{<T}^{(S)})\right\}, \quad (19)$$

where X_T is the input to Transformer block, S is a sequence length.

SLEB allows achieving 27% of acceleration with 8% of quality drop on LLaMA2-7B compared to the original model [78]. This method also requires extra inference on calibration data, leading to higher time costs.

Shortened-LLaMA [40] measures the significance of Transformer blocks by their impact on loss, assessing the error caused by the removal of a weight parameter (20).

$$i_T = L(W_T + \Delta W_T) - L(W_T) = \nabla_W L \Delta W_T, \quad (20)$$

where W_T is a Transformer block weight, L is a loss.

Shortened-LLaMA allows achieving 23% of acceleration with 7% of quality drop on LLaMA2-7B compared to the original model [40]. Notably, like *SLEB*, *Shortened-LLaMA* incurs additional time overhead due to the need for inference on the calibration dataset.

1.3.4 Importance of predicted architectures

Finally, the importance of the full architectures could be estimated. Such approaches are being developed within the field of Neural Architecture Search. Neural Architecture Search (NAS) is a technique for automating the design of neural networks. Instead of manually developing neural network architectures, NAS algorithms explore and discover optimal architectures for specific tasks.

For this purpose NAS utilizes a special controller to estimate the performance of a predicted neural architecture on a proxy dataset. However, this evaluation method, particularly when based on validation accuracy, can be computationally prohibitive. For example, NASnet [73] approach requires more than 2000 GPU days to identify optimal model architecture.

In response to the high computational demands of traditional NAS methods, zero-cost proxies present a more resource-efficient alternative. Zero-cost methods rely heavily on importance heuristics to approximate the performance of neural architectures without the need for extensive training. For example, the TF-TAS [101] approach utilizes synaptic diversity for the MHA module and synaptic saliency for the FFNN module. TF-TAS improves upon existing techniques, leading to a 4% quality improvement compared with manually designed ResNet and MobileNet-V3. At the same time, TF-TAS significantly reduces the number of GPU days required to estimate the predicted architecture compared with AutoFormer [10] and ViTAS [81].

A comprehensive comparison study of 15 zero-cost proxies applicable to Transformer architectures and LLMs [20] revealed varying levels of its effectiveness. These proxies were assessed through Kendall and Spearman correlations, with values ranging from -1 to 1, where higher values indicate better predictive accuracy of neural architecture rankings vis-à-vis fully trained models. Despite their utility, many proxies exhibited low correlation values, with Kendall's ranging from 0.01 to 0.5 and Spearman's from 0.1 to 0.5 [20].

Therefore, various structures of LLMs, including individual weights, layers, and

Transformer blocks, as well as entire model architectures, can be evaluated in terms of their importance, specifically regarding their impact on the overall performance of the model. Such importance assessment methods are actively utilized in different optimization strategies for LLM deployment during inference, such as sparsification and quantization. Among the structural components considered, the Transformer block is particularly noteworthy. Existing research has already demonstrated the potential for compressing LLM, achieving up to a 30% time and memory usage reduction by assessing the importance of Transformer blocks and retaining only the most significant ones. In contrast, approaches that focus on individual elements or layers have not achieved comparable resource savings, while methods focusing on entire architectures require substantial time and memory resources and utilized only within a Neural Architecture Search field. Hence, the analysis of Transformer block importance emerges as the most promising direction in this context, being particularly popular in the field of model inference.

However, while existing solutions for estimating the Transformer block importance successfully minimize computational costs, they often lead to a substantial quality degradation of LLM performance. As a step towards addressing this challenge, the assessment of the importance of Transformer blocks can be further utilized within a fine-tuning field, concurrently optimizing fine-tuning costs.

The next chapter presents a novel fine-tuning strategy, that dynamically allocates *LoRA* adapters to essential Transformer blocks based on efficient importance analysis to further accelerate the fine-tuning process, while preserving the model quality.

1.4 Overview of considered competitors

To validate the efficacy of *ID-LoRA*, a comparative analysis was employed with alternative approaches in the domain of fine-tuning acceleration and importance analysis of LLM layers and Transformer blocks.

Examined methods to analyze the importance of Transformer blocks include:

1. *UIDL* [26] approach. After estimating the most important model layers during the inference on calibration data and further pruning of the least essential structures, *UIDL* utilizes QLoRA fine-tuning to enhance model quality.
2. The *SLEB* [78] approach.
3. The *Shortened-LLaMA* [40] approach.
4. The *OWS* [51] approach. After estimating the importance during the inference on calibration data and further pruning of the least essential structures, *OWS* utilizes full fine-tuning to enhance model quality.
5. The *LISA* [71] approach, which randomly selects Transformer blocks during the process of full fine-tuning. This method does not rely on any deliberate heuristic to estimate the importance, that is why it can be considered as the reference approach for understanding the effect of the "importance".

Notably, all these approaches, except *LISA*, employ different heuristics to estimate Transformer block importance, which could provide insights into the effectiveness of existing metrics and their correlation with LLM performance. Also, it helps to evaluate whether using these importance measures can optimize the fine-tuning process compared to *LoRA*. For practical implementation, the official implementation of considered approaches was taken from corresponding Github repositories: *UIDL* [27], *SLEB* [79], *Shortened-LLaMA* [41], *OWS* [51], *LISA* [71].

Since *LoRA* is the cutting-edge approach for fine-tuning acceleration, it was selected as a baseline for computational experiments on quality and acceleration of considered approaches. In addition, the most prominent modifications of *LoRA*, such as *DoRA* [56], *PiSSA* [67], *AdaLoRA* [97], and *LISA* [71] were chosen as reference methods during the evaluation.

Chapter 2. Development of an approach to accelerate the fine-tuning through importance analysis

The discussed idea of estimating the importance of Transformer blocks formed the basis of the proposed method *ID-LoRA* (*Importance-driven LoRA*). *ID-LoRA* selectively assigns and re-assigns *LoRA* adapters based on Transformer blocks importance during the fine-tuning process with subsequent reduction of the number of Transformer blocks.

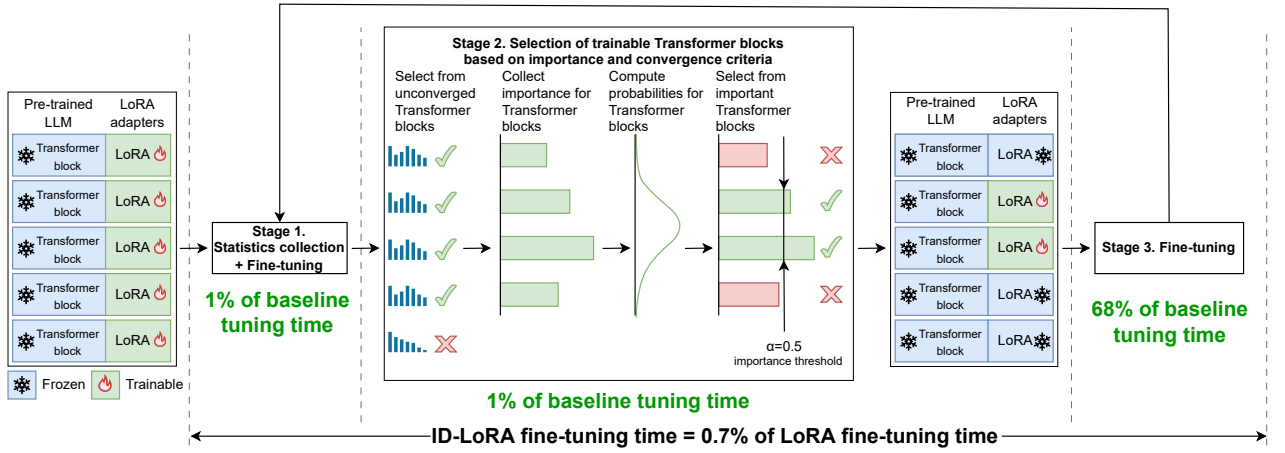


Figure 1: Key idea of proposed fine-tuning method: *Importance-driven LoRA*.

Let h denote history defined as the number of steps to collect importance and convergence statistics, α denote the ratio of Transformer blocks to be selected for fine-tuning.

Initially, the fine-tuning process is divided into several periods, during which the algorithm switches the trainable Transformer blocks. Each *ID-LoRA* period has 3 stages: statistics collection, selection of trainable Transformer blocks, and fine-tuning of the selected blocks. At the beginning of each stage, layer-wise importance statistic is collected during h steps. Next, algorithm determines unimportant Transformer blocks and removes them from the selection. After that, importance sampling of Transformer blocks is applied, and the ratio of the most important Transformer blocks (α) is selected for further fine-tuning. At the same time, *LoRA* A , B adapters for not-trainable Transformer blocks are merged up to the end of the fine-tuning. Merging

non-updatable adapters allows to further accelerate the fine-tuning process and increase memory reduction. Obviously, with each training period, the number of trainable Transformer blocks diminishes. As a result, fine-tuning stops once there are no more trainable Transformer blocks. As an alternative or a complementary strategy to importance analysis, a convergence analysis was proposed by team to further reduce the number of trainable Transformer blocks.

The full *ID-LoRA* method with importance analysis as my personal contribution and convergence analysis as team proposal is described in Algorithm 1.

2.1 Importance-based Transformer blocks selection

Statistics collection phase The first step of *ID-LoRA* is dedicated to statistics collection over several iterations of fine-tuning. During h steps, importance scores are accumulated for each layer of the model. As an importance estimation method several metrics have been considered, including Magnitude rule, Wanda rule, and Cosine Similarity rule. Notably, Magnitude [28] and Wanda [82] importance rules are adapted to Transformer block importance estimation as described in Definition 1.

Definition 1. Let T denote Transformer blocks, i_ξ denote importance score for ξ layer inside it, then importance of Transformer block is defined as:

$$I(T) = \frac{\sum_{\xi \in T} i_\xi}{|T|} \quad (21)$$

Selection phase Upon reaching the h number of steps, the importance scores of each Transformer block are averaged. After that, the second step of *ID-LoRA* begins, aimed at selecting important Transformer blocks. Based on the importance scores, selection probability for each Transformer block is computed (22). Next, the Top-P sampling (23) is applied, which samples Transformer blocks with the highest probability scores until the sum of the scores reaches the specified threshold value. Then, the sampling probability for each Transformer block is recalculated (22). Finally, α of

Algorithm 1 *ID-LoRA* (α, h)

```
1:  $D$ : fine-tuning dataset
2:  $\mathcal{M}_\Phi$ : pre-trained model
3:  $\alpha$ : trainable blocks selection ratio
4:  $h$ : number of accumulation steps
5:
6:  $C \leftarrow \{\}$ : converged Transformer blocks
7:  $\mathcal{Q} \leftarrow \{\mathcal{M}_\Phi\}$ : trainable Transformer blocks
8:  $I \leftarrow \{\}$ : accumulated importance scores
9:  $\mathcal{W} \leftarrow \{\}$ : accumulated layer weights
10:  $use\_convergence = True$ 
11: for epoch  $i = 1, \dots, N$  do
12:   for train step  $i = 1, \dots, K$ , global batch  $b \in D$  do
13:      $inp, labels \leftarrow b$ : data for forward
14:     if  $i < h$  then
15:       {Statistics Collection + Fine-tuning Phase}
16:       for layer  $\xi$  in  $\mathcal{M}_\Phi$  do
17:          $inp, out \leftarrow forward(\xi, inp)$ 
18:          $I \leftarrow I \cup i(\xi, inp, out)$ : calculate layer importance
19:         if  $use\_convergence$  then
20:            $\mathcal{W} \leftarrow \mathcal{W} \cup w_\xi$ 
21:         end if
22:          $grads \leftarrow backward(loss(out, labels))$ 
23:          $apply\_gradients(grads)$ 
24:       end for
25:     else if  $i = h$  then
26:       {Selection Phase}
27:       for  $\mathcal{T}$  in  $\mathcal{Q}$  do
28:          $I_\mathcal{T} \leftarrow I(\{w_j : w_j \in \mathcal{T}\})$ : aggregate importance of Transformer blocks (21)
29:         if  $use\_convergence$  then
30:            $C_\mathcal{T} \leftarrow c(\{w_i : w_i \in \mathcal{T}\})$ : calculate layer convergence
31:         end if
32:       end for
33:        $T \leftarrow \{\mathcal{T}_i \in \mathcal{Q} : C_{\mathcal{T}_i} \geq \epsilon\}$ 
34:        $P \leftarrow softmax(\{I_\mathcal{T} : \mathcal{T}_i \in T\})$ : select with (22)
35:        $T \leftarrow \alpha * top\_p(\mathcal{Q}, P)$ : (23)
36:        $Unfreeze(R)$ 
37:        $Freeze(\mathcal{Q} \setminus R)$ 
38:        $\mathcal{Q} \leftarrow R$ 
39:     else
40:       {Fine-tuning Phase}
41:        $inp, out \leftarrow forward(inp)$ 
42:        $grads \leftarrow backward(loss(out, labels))$ 
43:        $apply\_gradients(grads)$ 
44:     end if
45:   end for
46: end for
```

the most important Transformer blocks are selected for further fine-tuning.

$$P(T) = \frac{e^{I(T)}}{\sum_{k=1}^{\mathcal{M}_\Phi} e^{I(k)}}, \quad (22)$$

where I denotes Transformer block importance score.

$$Top_P = \{T_k \in \mathcal{M}_\Phi : \sum_{k=1}^{\mathcal{M}_\Phi} P(T_k) \leq p\}, \quad (23)$$

where p denotes the threshold.

2.2 Convergence-based Transformer blocks selection

Besides the importance analysis, another solution proposed by the team is an optional convergence analysis, which identifies fully trained Transformer blocks that could be frozen, thereby further reducing the number of trainable Transformer blocks. Once per h steps, additional statistics for convergence analysis are accumulated.

Definition 2. Let ε denote convergence threshold, $\hat{c}$ denote convergence metric. Then convergence rule for a layer is defined as:

$$c(W_\xi) = \begin{cases} 1, & \hat{c}(W_\xi) < \varepsilon \\ 0, & \text{otherwise} \end{cases}$$

Definition 3. Transformer block T is converged if the following inequality with convergence threshold δ is satisfied:

$$C(T) = \frac{\sum_{\xi \in T} c(W_\xi)}{|T|} > \delta \quad (24)$$

When statistic has been collected, the converged Transformer blocks are identified with the help of convergence rule (2) and corresponding metrics (3). After that, converged Transformer blocks are frozen up to the end of training, which leads to further reduction of the number of trainable parameters and an earlier stop of the fine-tuning process.

2.3 Key differences from existing approaches

- Current approaches utilize importance-based pruning of Transformer blocks on inference, introducing considerable quality degradation. In contrast, *ID-LoRA* integrates the pruning process into the parameter-efficient fine-tuning with *LoRA*, thereby preventing performance degradation.
- Prior methods incur extra computational costs by requiring inference on calibration data for importance estimation. Conversely, *ID-LoRA* identifies essential Transformer blocks during the fine-tuning process without additional overhead and calibration data.
- Current state-of-the-art approaches manually attach the *LoRA* adapters to all Transformer blocks before the fine-tuning process. On the contrary, *ID-LoRA* offers an innovative approach that enables automatic assignment of *LoRA A, B* adapters only to important Transformer blocks.
- *ID-LoRA* applies the idea of additional convergence analysis to further reduce the number of trainable Transformer blocks.

Chapter 3. Evaluation of the proposed method

To assess the feasibility of the *ID-LoRA*, comprehensive evaluation of the proposed solution has been applied with results being presented in the article "".

3.1 Training and evaluation protocols

For the evaluation purposes, six LLMs with different numbers of parameters were chosen to estimate the performance on different scales of LLMs:

- GPT2-M with 24 Transformer blocks;
- LLaMA2-7B with 32 Transformer blocks;
- LLaMA2-13B with 40 Transformer blocks;
- Qwen2.5-3B with 36 Transformer blocks;
- Qwen2.5-7B with 28 Transformer blocks;
- Qwen2.5-32B with 64 Transformer blocks.

The GPT2-M model was taken from *transformers* v4.33.2, LLaMA2-7B, LLaMA2-13B, Qwen2.5-3B, Qwen2.5-7B, Qwen2.5-32B models were taken from HuggingFace hub [5], [4], [75], [76], [74].

Firstly, for the *UIDL*, *SLEB*, *Shortened-LLaMA*, and *OWS* approaches, the importance estimation of Transformer blocks was conducted on a validation set of the Alpaca dataset. Subsequently, for the *UIDL* the QLoRA fine-tuning was applied to key Transformer blocks, while non-essential ones were left unchanged. In contrast, for *OWS* the full fine-tuning of critical units was utilized, while others were frozen. Originally, *SLEB* and *Shortened-LLaMA* do not employ fine-tuning to improve model performance. However, following the pruning of non-essential Transformer blocks, which involved their total removal, the *LoRA* fine-tuning method was utilized to

important Transformer blocks. The implementation of *LoRA*, *DoRA*, *PiSSA*, and *AdaLoRA* approaches was taken from the *peft v.0.18.0* library [64].

Table 2: Training hardware used for the evaluation of considered approaches.

	GPT2-M	LLaMA2-7B, LLaMA2-13B, Qwen2.5-3B, Qwen2.5-7B, Qwen2.5-32B
CPU	CPU 32 cores with 3.0GHz	CPU 32 cores with 2.90GHz
OS	Ubuntu 20.04.6 LTS	Ubuntu 20.04.04 LTS
Driver Version	515.43.04	535.183.01
Kernel Version	5.4.0-176-generic	5.15.0-125-generic
CUDA Toolkit Version	11.7.64	12.2
Training Libraries	<i>transformers</i> 4.33.0, <i>PyTorch</i> 2.1.0, <i>DeepSpeed</i> 0.15.4, <i>peft</i> v.0.18.0	<i>transformers</i> 4.45.0, <i>PyTorch</i> 2.1.0, <i>DeepSpeed</i> 0.15.4, <i>peft</i> v.0.18.0

Table 3: Training protocol used for the evaluation of considered approaches.

	GPT2-M	LLaMA2-7B, LLaMA2-13B, Qwen2.5-3B, Qwen2.5-7B, Qwen2.5-32B
Sequence Length	512	256
Learning Rate	6e-4	1e-5
AdamW β_1	0.9	0.9
AdamW β_2	0.99	0.99
Weight Decay	0.01	0
Epochs	2	2
Precision	fp16	bf16
<i>LoRA</i> r	16	16
<i>LoRA</i> α	32	32
Ratio of Transformer blocks	0.8	0.6

Used training hardware and software are described in Table 2. Training protocol for each model is described in Table 3. The *Ratio of Transformer blocks* row means the ratio of Transformer blocks selected for fine-tuning with chosen approaches. Notably, the GPT2-M model requires a higher number of updated layers than all other models, because a greater number of layers in the model allows for a smaller amount of

updated layers, since a substantial proportion of the layers will still be updated.

After the fine-tuning, in order to estimate the final quality of the model, zero-shot accuracy was measured on commonsense reasoning datasets: PIQA [6], HellaSwag [95], Winogrande [77], ARC [14], OpenbookQA [69], MMLU [33] using the lm-evaluation-harness package [24]. Zero-shot PPL was measured on language modelling dataset WikiText2 [68]. The detailed results for each of these datasets are presented in the paper.

3.2 Analysis of the obtained results

Computational experiments with chosen models and approaches are presented in Table 4 and Table 5. Average results of fine-tuning quality and acceleration for *ID-LoRA* without additional convergence analysis are presented in Table ??, Table ??, and Table 5. For LLaMA-family and Qwen-family models, the evaluation of *LISA* and *OVS* approaches was not presented, as memory requirements on the full fine-tuning of these models did not fit to used hardware.

ID-LoRA provides an average acceleration of 33.1% with a considerable maximum speedup of 44.8% for LLaMA2-13B with a quality decrease of less than 0.1%. In addition, *ID-LoRA* significantly outperforms *PiSSA*, *DoRA*, and *AdaLoRA* from the point of acceleration, and provides 0.8% of quality improvement compared to *AdaLoRA*. However, it leads to 0.1% and 1.6% quality decrease compared to *DoRA* and *PiSSA* respectively as *PiSSA* employs SVD initialization of *LoRA* *A*, *B* adapters and *DoRA* incorporates additional magnitude vector during the fine-tuning.

Performance analysis of *ID-LoRA* compared to the such state-of-the-art pruning approaches as *UIDL*, *SLEB*, and *Shortened-LLaMA* revealed, that *ID-LoRA* provides 1.1%, 43.8%, and 79.6% quality improvement compared to *UIDL*, *SLEB*, and *Shortened-LLaMA*, respectively, and outperforms all these methods from the point of acceleration.

Table 4: Quality results after the fine-tuning with considered approaches on GPT2-M, LLaMA2-7B, and LLaMA2-13B.

Model	Method	Average accuracy, $\uparrow$	Perplexity, $\downarrow$	Acceleration, $\uparrow$
GPT2-M	<i>LoRA</i> (baseline)	37.5	173.1	–
	<i>DoRA</i>	37.0	184.2	-21.6
	<i>PiSSA</i>	34.9	172.1	0.9
	<i>AdaLoRA</i>	36.9	179.2	-1.3
	<i>UIDL</i>	37.9	173.8	-39.5
	<i>LISA</i>	40.8	205.7	-40.9
	<i>OWS</i>	31.3	180.2	-61.3
	<i>SLEB</i>	36.3	450.1	13.5
	<i>Shortened-LLaMA</i>	33.1	400.5	12.6
	<i>ID-LoRA</i>	38.0	72.7	25.2
LLaMA2-7B	<i>LoRA</i> (baseline)	59.9	9.2	–
	<i>DoRA</i>	59.8	9.2	-407.3
	<i>PiSSA</i>	60.9	9.4	1.7
	<i>AdaLoRA</i>	59.3	9.2	-51.6
	<i>UIDL</i>	59.3	9.2	-31.2
	<i>SLEB</i>	42.7	403.6	-11.0
	<i>Shortened-LLaMA</i>	41.1	53.1	13.4
	<i>ID-LoRA</i>	59.4	9.2	29.2
LLaMA2-13B	<i>LoRA</i> (baseline)	63.8	7.7	–
	<i>DoRA</i>	63.7	7.7	-203.3
	<i>PiSSA</i>	64.5	7.9	-0.3
	<i>AdaLoRA</i>	63.1	7.7	-92.9
	<i>UIDL</i>	63.6	7.7	-22.7
	<i>SLEB</i>	49.8	20.0	-70.0
	<i>Shortened-LLaMA</i>	42.4	72.8	-110.7
	<i>ID-LoRA</i>	63.6	7.7	44.8
Qwen2.5-3B	<i>LoRA</i> (baseline)	62.9	15.5	–
	<i>DoRA</i>	62.9	15.3	-201.0
	<i>PiSSA</i>	63.5	11.8	-0.6
	<i>AdaLoRA</i>	62.3	15.1	-71.5
	<i>UIDL</i>	62.7	15.5	-20.1
	<i>SLEB</i>	44.5	77.4	-30.4
	<i>Shortened-LLaMA</i>	15.1	72.8	0.1
	<i>ID-LoRA</i>	62.9	15.5	28.6

Table 5: Quality results after the fine-tuning with considered approaches on Qwen2.5-7B and Qwen2.5-32B.

Model	Method	Average accuracy, $\uparrow$	Perplexity, $\downarrow$	Acceleration, $\uparrow$
Qwen2.5-7B	<i>LoRA</i> (baseline)	66.6	8.8	–
	<i>DoRA</i>	66.6	8.8	-761.3
	<i>PiSSA</i>	66.7	9.0	0.7
	<i>AdaLoRA</i>	66.4	8.7	-42.6
	<i>UIDL</i>	66.4	8.8	-31.2
	<i>SLEB</i>	42.1	40.3	-29.7
	<i>Shortened-LLaMA</i>	34.6	542.3	10.0
	<i>ID-LoRA</i>	66.5	8.7	33.2
Qwen2.5-32B	<i>LoRA</i> (baseline)	70.0	6.0	–
	<i>PiSSA</i>	70.8	6.1	-1.0
	<i>AdaLoRA</i>	69.6	6.0	-92.5
	<i>UIDL</i>	70.0	6.0	-25.4
	<i>SLEB</i>	56.2	18.7	-84.4
	<i>Shortened-LLaMA</i>	44.7	250.8	-81.3
	<i>ID-LoRA</i>	69.6	5.6	37.5

3.3 Sensitivity analysis

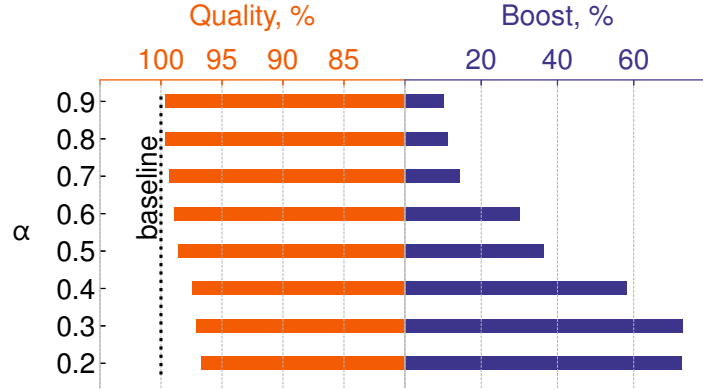


Figure 2: Sensitivity analysis of *ID-LoRA* to the α hyperparameter. h is 10, importance metric is cosine similarity.

Ablation study of the α hyperparameter of *ID-LoRA* is presented in Figure 2. It was analyzed in terms of their impact on the resulting quality and acceleration. Obtained results demonstrates that increasing α significantly affects both acceleration

and model quality. In particular, a maximum speedup of 72.6% is observed at $\alpha = 0.2$, where only 20% of the Transformer blocks are trained. However, it leads to a 3.3% increase in perplexity, suggesting underfitting. Minimal quality degradation (0.3%) occurs at higher α values (0.8 and 0.9), where 80-90% of the Transformer blocks are trained initially.

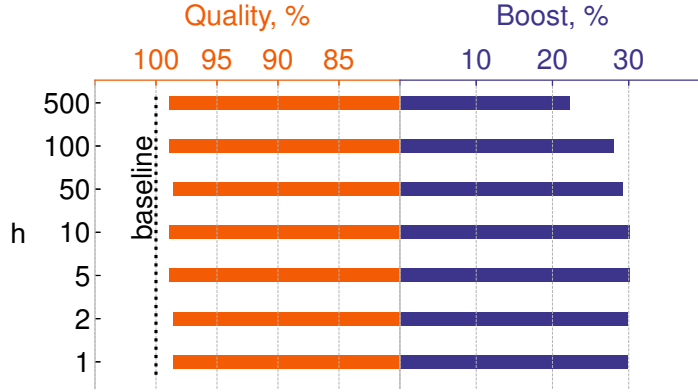


Figure 3: Sensitivity analysis of *ID-LoRA* to the h hyperparameter. α is 0.6, importance metric is cosine similarity.

Ablation study of the h hyperparameter of *ID-LoRA* is presented in Figure 3. It was analyzed in terms of their impact on the resulting quality and acceleration. According to the achieved results, variations do not significantly impact quality, with only a 0.3% change in perplexity was observed between $h = 1$ and $h = 500$. The smallest h value accelerates fine-tuning by 29.8%, indicating that even a single training step can adequately capture the importance of a Transformer block. This observation is supported by the relative stability of model quality across different h settings, as shown in Figure 4b. $h = 10$ is identified as the optimal setting for balancing speed and quality.

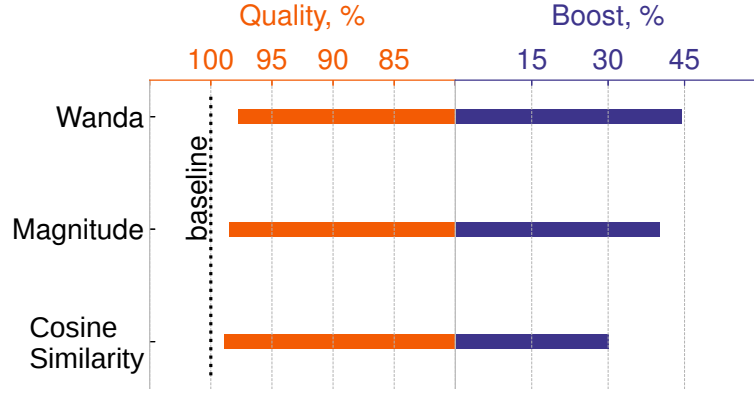


Figure 4: Sensitivity analysis of *ID-LoRA* to considered importance metrics. α is 0.6, h is 10.

Ablation study of importance metrics is presented in Figure 4. It reveals that the Magnitude metric yields the best balance between acceleration and quality. In contrast, the Wanda metric, despite considering input activations, provides a higher speedup at the cost of increased quality degradation by 2% compared to the baseline. Although cosine similarity decelerates the fine-tuning on 29.6% compared with the Magnitude, it degrades model quality the least, representing the most suitable choice, as minimizing the drop in model quality is particularly crucial in the context of importance-based fine-tuning.

Conclusion

This study was aimed to address to primary objectives:

- Provide an in-depth examination of existing importance estimation techniques across various scenarios, with a particular focus on the importance estimation of Transformer blocks to optimize the fine-tuning process.
- Introduce a novel fine-tuning approach for LLMs, that dynamically assigns *LoRA* adapters during the fine-tuning based on importance and convergence analysis of Transformer blocks.

Throughout the research, various modern approaches for reducing time and memory overhead during the fine-tuning were considered with *LoRA* de facto being the state-of-the-art solution in the field. However, it was revealed, that *LoRA* manually attaches low-parameterized adapters to the whole model by a user-defined mask, leading to computational overhead due to the LLMs overparametrization. Moreover, the concept of "importance" was deemed as the identification of structures that can be removed without significantly affecting model performance. It was found, that many of existing importance-based compression techniques leads to considerable quality degradation and provide substantial costs due to additional inference on calibration data.

To address the aforementioned challenges, in the second part of this study, a novel *LoRA*-based fine-tuning approach, *ID-LoRA*, that utilizes importance estimation for efficient reallocation of *LoRA* adapters has been proposed. To thoroughly evaluate the proposed method *ID-LoRA* a performance and comparison analysis on different LLMs and datasets has been applied. As a baseline, a cutting-edge *LoRA* approach has been used. In addition, key Transformer blocks-based importance approaches like *UIDL*, *OWS*, *SLEB*, *LISA*, and *Shortened-LLaMA* and the most prominent modifications of *LoRA* – *DoRA*, *PiSSA*, and *AdaLoRA* – were chosen for reference.

The 6 models' fine-tuning was estimated with considered approaches, including

GPT2-M, LLaMA2-7B, LLaMA2-13B, Qwen2.5-3B, Qwen2.5-7B, and Qwen2.5-32B. After the fine-tuning, in order to estimate the final quality of the model, zero-shot accuracy was measured on commonsense reasoning datasets: PIQA [6], HellaSwag [95], Winogrande [77], ARC [14], OpenbookQA [69], MMLU [33] using the lm-evaluation-harness package [24]. Zero-shot PPL was measured on language modelling dataset WikiText2 [68].

The results showed that:

- Proposed *ID-LoRA* demonstrates a notable acceleration of 53% for Qwen2.5-3B, 59% for Qwen2.5-7B, 50% for LLaMA2-7B, 54% for LLaMA2-13B, and 49% for Qwen2.5-32B without sacrificing the accuracy.
- Proposed *ID-LoRA* significantly outperforms all considered reference importance estimation and fine-tuning acceleration approaches from the point of fine-tuning time acceleration while preserving the quality.

Therefore, future directions of this study within research project in Huawei include:

1. Provide a theoretical justification for the effectiveness of the proposed *ID-LoRA*.
2. Conduct an analysis, whether entirely freezing Transformer blocks during the fine-tuning may be imprudent, since it is possible that temporarily removing them from the fine-tuning with subsequent reactivation could provide quality improvement.
3. Explore the interoperability of *ID-LoRA* with *LoRA* modifications that could lead to mixed results when using *ID-LoRA* with other *LoRA*-based approaches.

References

- [1] Z. Allen-Zhu, Y. Li, and Z. Song, “A convergence theory for deep learning via over-parameterization,” 2019. [Online]. Available: <https://arxiv.org/abs/1811.03962>
- [2] Y. An, X. Zhao, T. Yu, M. Tang, and J. Wang, “Systematic outliers in large language models,” 2025. [Online]. Available: <https://arxiv.org/abs/2502.06415>
- [3] S. Arora, N. Cohen, N. Golowich, and W. Hu, “A convergence analysis of gradient descent for deep linear neural networks,” 2019. [Online]. Available: <https://arxiv.org/abs/1810.02281>
- [4] D. Autar, “daryl149/llama-2-13b-chat-hf. Hugging Face — huggingface.co,” <https://huggingface.co/daryl149/llama-2-13b-chat-hf>, 2023, [Accessed 20-04-2025].
- [5] —, “daryl149/llama-2-7b-hf. Hugging Face — huggingface.co,” <https://huggingface.co/daryl149/llama-2-7b-hf>, 2023, [Accessed 20-04-2025].
- [6] Y. Bisk, R. Zellers, R. L. Bras, J. Gao, and Y. Choi, “Piqa: Reasoning about physical commonsense in natural language,” 2019. [Online]. Available: <https://arxiv.org/abs/1911.11641>
- [7] R. Brown, “Donald o. hebb and the organization of behavior: 17 years in the writing,” *Molecular Brain*, vol. 13, 12 2020.
- [8] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray,

- B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, “Language models are few-shot learners,” 2020. [Online]. Available: <https://arxiv.org/abs/2005.14165>
- [9] G. Cazenavette, T. Wang, A. Torralba, A. A. Efros, and J.-Y. Zhu, “Dataset distillation by matching training trajectories,” 2022. [Online]. Available: <https://arxiv.org/abs/2203.11932>
- [10] M. Chen, H. Peng, J. Fu, and H. Ling, “Autoformer: Searching transformers for visual recognition,” 2021. [Online]. Available: <https://arxiv.org/abs/2107.00651>
- [11] T. Chen, J. Frankle, S. Chang, S. Liu, Y. Zhang, Z. Wang, and M. Carbin, “The lottery ticket hypothesis for pre-trained bert networks,” 2020. [Online]. Available: <https://arxiv.org/abs/2007.12223>
- [12] X. Chen, Y. Hu, J. Zhang, Y. Wang, C. Li, and H. Chen, “Streamlining redundant layers to compress large language models,” 2025. [Online]. Available: <https://arxiv.org/abs/2403.19135>
- [13] H. Cheng, M. Zhang, and J. Q. Shi, “A survey on deep neural network pruning-taxonomy, comparison, analysis, and recommendations,” 2024. [Online]. Available: <https://arxiv.org/abs/2308.06767>
- [14] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord, “Think you have solved question answering? try arc, the ai2 reasoning challenge,” 2018. [Online]. Available: <https://arxiv.org/abs/1803.05457>
- [15] B. Cottier, R. Rahman, L. Fattorini, N. Maslej, T. Besiroglu, and D. Owen, “The rising costs of training frontier ai models,” 2025. [Online]. Available: <https://arxiv.org/abs/2405.21015>

- [16] A. Demidovskij, A. Tugaryov, I. Salnikov, O. Frolova, A. Trutnev, P. Xie, I. Novikova, E. Zharikov, V. Blyudova, and Y. Ignatiev, “Hebb-inspired low rank adapters for large language models fine-tuning,” in *PRICAI 2025: Trends in Artificial Intelligence*, Y. Mei, C. Qian, Q. Bai, B. Xue, and S. Khanna, Eds. Singapore: Springer Nature Singapore, 2026, pp. 603–612.
- [17] M. Denil, B. Shakibi, L. Dinh, M. Ranzato, and N. de Freitas, “Predicting parameters in deep learning,” 2014. [Online]. Available: <https://arxiv.org/abs/1306.0543>
- [18] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, “Llm.int8(): 8-bit matrix multiplication for transformers at scale,” 2022. [Online]. Available: <https://arxiv.org/abs/2208.07339>
- [19] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “Qlora: Efficient finetuning of quantized llms,” 2023. [Online]. Available: <https://arxiv.org/abs/2305.14314>
- [20] P. Dong, L. Li, X. Liu, Z. Tang, X. Liu, Q. Wang, and X. Chu, “Lpzero: Language model zero-cost proxy search from zero,” 2024. [Online]. Available: <https://arxiv.org/abs/2410.04808>
- [21] A. Draganov, S. Vadgama, and E. J. Bekkers, “The hidden pitfalls of the cosine similarity loss,” 2024. [Online]. Available: <https://arxiv.org/abs/2406.16468>
- [22] U. Endriss, F. S. Melo, K. Bach, A. J. B. Diz, J. M. Alonso-Moral, S. Barro, and F. Heintz, Eds., *ECAI 2024 - 27th European Conference on Artificial Intelligence, 19-24 October 2024, Santiago de Compostela, Spain - Including 13th Conference on Prestigious Applications of Intelligent Systems (PAIS 2024)*, ser. Frontiers in Artificial Intelligence and Applications, vol. 392. IOS Press, 2024. [Online]. Available: <https://doi.org/10.3233/FAIA392>

- [23] J. Frankle and M. Carbin, “The lottery ticket hypothesis: Finding sparse, trainable neural networks,” 2019. [Online]. Available: <https://arxiv.org/abs/1803.03635>
- [24] L. Gao, J. Tow, B. Abbasi, S. Biderman, S. Black, A. DiPofi, C. Foster, L. Golding, J. Hsu, A. Le Noac’h, H. Li, K. McDonell, N. Muennighoff, C. Ociepa, J. Phang, L. Reynolds, H. Schoelkopf, A. Skowron, L. Sutawika, E. Tang, A. Thite, B. Wang, K. Wang, and A. Zou, “A framework for few-shot language model evaluation,” 07 2024. [Online]. Available: <https://zenodo.org/records/12608602>
- [25] J. Gou, B. Yu, S. J. Maybank, and D. Tao, “Knowledge distillation: A survey,” *International Journal of Computer Vision*, vol. 129, no. 6, p. 1789–1819, Mar. 2021. [Online]. Available: <http://dx.doi.org/10.1007/s11263-021-01453-z>
- [26] A. Gromov, K. Tirumala, H. Shapourian, P. Gloriosi, and D. A. Roberts, “The unreasonable ineffectiveness of the deeper layers,” 2025. [Online]. Available: <https://arxiv.org/abs/2403.17887>
- [27] —, “The unreasonable ineffectiveness of the deeper layers,” <https://github.com/arcee-ai/PruneMe>, 2025, [Accessed 22-06-2025].
- [28] S. Han, H. Mao, and W. J. Dally, “Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding,” 2016. [Online]. Available: <https://arxiv.org/abs/1510.00149>
- [29] S. Han, J. Pool, J. Tran, and W. J. Dally, “Learning both weights and connections for efficient neural networks,” 2015. [Online]. Available: <https://arxiv.org/abs/1506.02626>
- [30] Z. Han, C. Gao, J. Liu, J. Zhang, and S. Q. Zhang, “Parameter-efficient fine-tuning for large models: A comprehensive survey,” 2024. [Online]. Available: <https://arxiv.org/abs/2403.14608>

- [31] J. He, S. Wu, W. Wen, C. J. Xue, and Q. Li, “Chess: Optimizing llm inference via channel-wise thresholding and selective sparsification,” 2024. [Online]. Available: <https://arxiv.org/abs/2409.01366>
- [32] S. He, G. Sun, Z. Shen, and A. Li, “What matters in transformers? not all attention is needed,” 2024. [Online]. Available: <https://arxiv.org/abs/2406.15786>
- [33] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, “Measuring massive multitask language understanding,” 2021. [Online]. Available: <https://arxiv.org/abs/2009.03300>
- [34] G. Hinton, “The forward-forward algorithm: Some preliminary investigations,” 2022. [Online]. Available: <https://arxiv.org/abs/2212.13345>
- [35] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “Lora: Low-rank adaptation of large language models,” 2021. [Online]. Available: <https://arxiv.org/abs/2106.09685>
- [36] J. hun Shim, K. Kong, and S.-J. Kang, “Core-set sampling for efficient neural architecture search,” 2021. [Online]. Available: <https://arxiv.org/abs/2107.06869>
- [37] A. H. Jiang, D. L. K. Wong, G. Zhou, D. G. Andersen, J. Dean, G. R. Ganger, G. Joshi, M. Kaminsky, M. Kozuch, Z. C. Lipton, and P. Pillai, “Accelerating deep learning by focusing on the biggest losers,” 2019. [Online]. Available: <https://arxiv.org/abs/1910.00762>
- [38] T. B. Johnson and C. Guestrin, “Training deep models faster with robust, approximate importance sampling,” in *Advances in Neural Information Processing Systems*, S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett, Eds., vol. 31. Curran Associates, Inc., 2018.

- [39] A. Katharopoulos and F. Fleuret, “Biased importance sampling for deep neural network training,” 2017. [Online]. Available: <https://arxiv.org/abs/1706.00043>
- [40] B.-K. Kim, G. Kim, T.-H. Kim, T. Castells, S. Choi, J. Shin, and H.-K. Song, “Shortened llama: Depth pruning for large language models with comparison of retraining methods,” 2024. [Online]. Available: <https://arxiv.org/abs/2402.02834>
- [41] —, “Shortened llama: Depth pruning for large language models with comparison of retraining methods,” <https://github.com/Nota-NetsPresso/shortened-llm>, 2024, [Accessed 22-06-2025].
- [42] J. Kim, J. H. Lee, S. Kim, J. Park, K. M. Yoo, S. J. Kwon, and D. Lee, “Memory-efficient fine-tuning of compressed large language models via sub-4-bit integer quantization,” 2023. [Online]. Available: <https://arxiv.org/abs/2305.14152>
- [43] S. Kim, C. Hooper, A. Gholami, Z. Dong, X. Li, S. Shen, M. W. Mahoney, and K. Keutzer, “Squeezellm: Dense-and-sparse quantization,” 2024. [Online]. Available: <https://arxiv.org/abs/2306.07629>
- [44] —, “Squeezellm: Dense-and-sparse quantization,” 2024. [Online]. Available: <https://arxiv.org/abs/2306.07629>
- [45] O. Kovaleva, S. Kulshreshtha, A. Rogers, and A. Rumshisky, “Bert busters: Outlier dimensions that disrupt transformers,” 2021. [Online]. Available: <https://arxiv.org/abs/2105.06990>
- [46] S. Krithivasan, S. Sen, S. Venkataramani, and A. Raghunathan, “Accelerating dnn training through selective localized learning,” *Frontiers in Neuroscience*, vol. 15, 01 2022.
- [47] S. J. Kwon, J. Kim, J. Bae, K. M. Yoo, J.-H. Kim, B. Park, B. Kim, J.-W. Ha,

- N. Sung, and D. Lee, “Alphatuning: Quantization-aware parameter-efficient adaptation of large-scale pre-trained language models,” 2022. [Online]. Available: <https://arxiv.org/abs/2210.03858>
- [48] G. Lagani, F. Falchi, C. Gennaro, and G. Amato, “Hebbian semi-supervised learning in a sample efficiency setting,” *Neural Networks*, vol. 143, p. 719–731, Nov. 2021. [Online]. Available: <http://dx.doi.org/10.1016/j.neunet.2021.08.003>
- [49] N. Lee, T. Ajanthan, and P. H. S. Torr, “Snip: Single-shot network pruning based on connection sensitivity,” 2019. [Online]. Available: <https://arxiv.org/abs/1810.02340>
- [50] P. Li, L. Yin, X. Gao, and S. Liu, “Outlier-weighted layerwise sampling for llm fine-tuning,” <https://github.com/pixeli99/OWS>, 2024, [Accessed 22-06-2025].
- [51] —, “Outlier-weighted layerwise sampling for llm fine-tuning,” 2025. [Online]. Available: <https://arxiv.org/abs/2405.18380>
- [52] —, “Outlier-weighted layerwise sampling for llm fine-tuning,” 2025. [Online]. Available: <https://arxiv.org/abs/2405.18380>
- [53] X. Li, Y. Yao, X. Jiang, X. Fang, X. Meng, S. Fan, P. Han, J. Li, L. Du, B. Qin, Z. Zhang, A. Sun, and Y. Wang, “Flm-101b: An open llm and how to train it with \$100k budget,” 2025. [Online]. Available: <https://arxiv.org/abs/2309.03852>
- [54] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, “Awq: Activation-aware weight quantization for llm compression and acceleration,” 2024. [Online]. Available: <https://arxiv.org/abs/2306.00978>

- [55] H. Liu, Z. Dai, D. R. So, and Q. V. Le, “Pay attention to mlps,” 2021. [Online]. Available: <https://arxiv.org/abs/2105.08050>
- [56] S.-Y. Liu, C.-Y. Wang, H. Yin, P. Molchanov, Y.-C. F. Wang, K.-T. Cheng, and M.-H. Chen, “Dora: Weight-decomposed low-rank adaptation,” 2024. [Online]. Available: <https://arxiv.org/abs/2402.09353>
- [57] Y. Liu, A. Singh, C. D. Freeman, J. D. Co-Reyes, and P. J. Liu, “Improving large language model fine-tuning for solving math problems,” 2023. [Online]. Available: <https://arxiv.org/abs/2310.10047>
- [58] Y. Liu, S. Agarwal, and S. Venkataraman, “Autofreeze: Automatically freezing model blocks to accelerate fine-tuning,” 2021. [Online]. Available: <https://arxiv.org/abs/2102.01386>
- [59] Z. Liu, B. Oguz, C. Zhao, E. Chang, P. Stock, Y. Mehdad, Y. Shi, R. Krishnamoorthi, and V. Chandra, “Llm-qat: Data-free quantization aware training for large language models,” 2023. [Online]. Available: <https://arxiv.org/abs/2305.17888>
- [60] K. Lv, Y. Yang, T. Liu, Q. Gao, Q. Guo, and X. Qiu, “Full parameter fine-tuning for large language models with limited resources,” 2024. [Online]. Available: <https://arxiv.org/abs/2306.09782>
- [61] X. Ma, G. Fang, and X. Wang, “Llm-pruner: On the structural pruning of large language models,” 2023. [Online]. Available: <https://arxiv.org/abs/2305.11627>
- [62] S. Malladi, T. Gao, E. Nichani, A. Damian, J. D. Lee, D. Chen, and S. Arora, “Fine-tuning language models with just forward passes,” 2024. [Online]. Available: <https://arxiv.org/abs/2305.17333>

- [63] ———, “Fine-tuning language models with just forward passes,” 2024. [Online]. Available: <https://arxiv.org/abs/2305.17333>
- [64] S. Mangrulkar, S. Gugger, L. Debut, Y. Belkada, S. Paul, and B. Bossan, “Peft: State-of-the-art parameter-efficient fine-tuning methods,” <https://github.com/huggingface/peft>, 2022.
- [65] N. Maslej, L. Fattorini, R. Perrault, Y. Gil, V. Parli, N. Kariuki, E. Capstick, A. Reuel, E. Brynjolfsson, J. Etchemendy, K. Ligett, T. Lyons, J. Manyika, J. C. Niebles, Y. Shoham, R. Wald, T. Walsh, A. Hamrah, L. Santarlaschi, J. B. Lotufo, A. Rome, A. Shi, and S. Oak, “Artificial intelligence index report 2025,” 2025. [Online]. Available: <https://arxiv.org/abs/2504.07139>
- [66] X. Men, M. Xu, Q. Zhang, B. Wang, H. Lin, Y. Lu, X. Han, and W. Chen, “Shortgpt: Layers in large language models are more redundant than you expect,” 2024. [Online]. Available: <https://arxiv.org/abs/2403.03853>
- [67] F. Meng, Z. Wang, and M. Zhang, “Pissa: Principal singular values and singular vectors adaptation of large language models,” 2025. [Online]. Available: <https://arxiv.org/abs/2404.02948>
- [68] S. Merity, C. Xiong, J. Bradbury, and R. Socher, “Pointer sentinel mixture models,” 2016. [Online]. Available: <https://arxiv.org/abs/1609.07843>
- [69] T. Mihaylov, P. Clark, T. Khot, and A. Sabharwal, “Can a suit of armor conduct electricity? a new dataset for open book question answering,” 2018. [Online]. Available: <https://arxiv.org/abs/1809.02789>
- [70] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe, “Training language models to follow instructions with human feedback,” 2022. [Online]. Available: <https://arxiv.org/abs/2203.02155>

- [71] R. Pan, X. Liu, S. Diao, R. Pi, J. Zhang, C. Han, and T. Zhang, “Lisa: Layerwise importance sampling for memory-efficient large language model fine-tuning,” 2024. [Online]. Available: <https://arxiv.org/abs/2403.17919>
- [72] —, “Lisa: Layerwise importance sampling for memory-efficient large language model fine-tuning,” <https://github.com/osehmathias/lisa>, 2024, [Accessed 22-06-2025].
- [73] X. Qin and Z. Wang, “Nasnet: A neuron attention stage-by-stage net for single image deraining,” 2020. [Online]. Available: <https://arxiv.org/abs/1912.03151>
- [74] Qwen, “Qwen/qwen2.5-32b. Hugging Face — huggingface.co,” <https://huggingface.co/Qwen/Qwen2.5-32B>, 2024, [Accessed 20-04-2025].
- [75] —, “Qwen/qwen2.5-3b. Hugging Face — huggingface.co,” <https://huggingface.co/Qwen/Qwen2.5-3B>, 2024, [Accessed 20-04-2025].
- [76] —, “Qwen/qwen2.5-7b. Hugging Face — huggingface.co,” <https://huggingface.co/Qwen/Qwen2.5-7B>, 2024, [Accessed 20-04-2025].
- [77] K. Sakaguchi, R. L. Bras, C. Bhagavatula, and Y. Choi, “Winogrande: An adversarial winograd schema challenge at scale,” 2019. [Online]. Available: <https://arxiv.org/abs/1907.10641>
- [78] J. Song, K. Oh, T. Kim, H. Kim, Y. Kim, and J.-J. Kim, “Sleb: Streamlining llms through redundancy verification and elimination of transformer blocks,” 2024. [Online]. Available: <https://arxiv.org/abs/2402.09025>
- [79] —, “Sleb: Streamlining llms through redundancy verification and elimination of transformer blocks,” <https://github.com/jiwonsong-dev/SLEB>, 2025, [Accessed 22-06-2025].
- [80] J. Spall, “Multivariate stochastic approximation using a simultaneous

- perturbation gradient approximation,” *IEEE Transactions on Automatic Control*, vol. 37, no. 3, pp. 332–341, 1992.
- [81] X. Su, S. You, J. Xie, M. Zheng, F. Wang, C. Qian, C. Zhang, X. Wang, and C. Xu, “Vitas: Vision transformer architecture search,” 2021. [Online]. Available: <https://arxiv.org/abs/2106.13700>
 - [82] M. Sun, Z. Liu, A. Bair, and J. Z. Kolter, “A simple and effective pruning approach for large language models,” 2024. [Online]. Available: <https://arxiv.org/abs/2306.11695>
 - [83] H. Tanaka, D. Kunin, D. L. K. Yamins, and S. Ganguli, “Pruning neural networks without any data by iteratively conserving synaptic flow,” 2020. [Online]. Available: <https://arxiv.org/abs/2006.05467>
 - [84] Y. Tian, Y. Wang, Z. Zhang, B. Chen, and S. Du, “Joma: Demystifying multilayer transformers via joint dynamics of mlp and attention,” 2024. [Online]. Available: <https://arxiv.org/abs/2310.00535>
 - [85] A. Tugaryov, A. Trutnev, A. Demidovskij, and I. Salnikov, “Aloe: Boosting large language model fine-tuning with aggressive loss-based elimination of samples,” 10 2024.
 - [86] T. Tuncer, F. Ertam, S. Dogan, E. Aydemir, and P. Pławiak, “Ensemble residual networks based gender and activity recognition method with signals,” *The Journal of Supercomputing*, vol. 76, p. 2119–2138, 03 2020.
 - [87] G. Tyukin, G. J.-S. Dovonon, J. Kaddour, and P. Minervini, “Attention is all you need but you don’t need all of it for inference of large language models,” 2024. [Online]. Available: <https://arxiv.org/abs/2407.15516>
 - [88] E. Voita, D. Talbot, F. Moiseev, R. Sennrich, and I. Titov, “Analyzing

- multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned,” 2019. [Online]. Available: <https://arxiv.org/abs/1905.09418>
- [89] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” 2023. [Online]. Available: <https://arxiv.org/abs/2201.11903>
- [90] G. Wu and H.-I. Suk, “Deep learning in medical image analysis,” *Annual review of biomedical engineering*, vol. 19, 03 2017.
- [91] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, “Smoothquant: Accurate and efficient post-training quantization for large language models,” 2024. [Online]. Available: <https://arxiv.org/abs/2211.10438>
- [92] L. Xu, H. Xie, S.-Z. J. Qin, X. Tao, and F. L. Wang, “Parameter-efficient fine-tuning methods for pretrained language models: A critical review and assessment,” 2023. [Online]. Available: <https://arxiv.org/abs/2312.12148>
- [93] L. Yin, Y. Wu, Z. Zhang, C.-Y. Hsieh, Y. Wang, Y. Jia, G. Li, A. Jaiswal, M. Pechenizkiy, Y. Liang, M. Bendersky, Z. Wang, and S. Liu, “Outlier weighed layerwise sparsity (owl): A missing secret sauce for pruning llms to high sparsity,” 2024. [Online]. Available: <https://arxiv.org/abs/2310.05175>
- [94] M. Yu, D. Wang, Q. Shan, C. Reed, and A. Wan, “The super weight in large language models,” 2024. [Online]. Available: <https://arxiv.org/abs/2411.07191>
- [95] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi, “Hellaswag: Can a machine really finish your sentence?” 2019. [Online]. Available: <https://arxiv.org/abs/1905.07830>
- [96] C. Zhang, S. Bengio, and Y. Singer, “Are all layers created equal?” 2022. [Online]. Available: <https://arxiv.org/abs/1902.01996>

- [97] Q. Zhang, M. Chen, A. Bukharin, N. Karampatziakis, P. He, Y. Cheng, W. Chen, and T. Zhao, “Adalora: Adaptive budget allocation for parameter-efficient fine-tuning,” 2023. [Online]. Available: <https://arxiv.org/abs/2303.10512>
- [98] Y. Zhang, P. Li, J. Hong, J. Li, Y. Zhang, W. Zheng, P.-Y. Chen, J. D. Lee, W. Yin, M. Hong, Z. Wang, S. Liu, and T. Chen, “Revisiting zeroth-order optimization for memory-efficient llm fine-tuning: A benchmark,” 2024. [Online]. Available: <https://arxiv.org/abs/2402.11592>
- [99] A. Zhou, Y. Ma, J. Zhu, J. Liu, Z. Zhang, K. Yuan, W. Sun, and H. Li, “Learning n:m fine-grained structured sparse neural networks from scratch,” 2021. [Online]. Available: <https://arxiv.org/abs/2102.04010>
- [100] —, “Learning n:m fine-grained structured sparse neural networks from scratch,” 2021. [Online]. Available: <https://arxiv.org/abs/2102.04010>
- [101] Q. Zhou, K. Sheng, X. Zheng, K. Li, X. Sun, Y. Tian, J. Chen, and R. Ji, “Training-free transformer architecture search,” 2022. [Online]. Available: <https://arxiv.org/abs/2203.12217>